

Commsignia Safety Applications for Micromobility Devices

Document ID: SAF-001-M

Version: 25-02

Commsignia Ltd. reserves all rights to this document and the information contained herein. Products, names, logos and designs described may in whole or in part be subject to intellectual property rights.

Confidentiality notice: This document is provided in confidence and may not be used for any purpose other than that for which it is supplied. All content within may not be disclosed to any third party or used for other purpose without the written permission of Commsignia Ltd.

This document may be revised without prior notice. For most recent releases, please visit our website.

Publication date February 14, 2025

Copyright © 2025 Commsignia Ltd.

Table of Contents

1. Introduction	1
2. Common notification contents	3
2.1. Object type	3
2.2. Vehicle type	3
2.3. Object data	3
2.4. Relative data	4
3. Speed-dependent behavior	5
3.1. Speed-dependent configuration	5
3.2. Logic of speed-dependent behavior	5
4. Safety applications	6
4.1. I2M Applications	6
4.1.1. Pedestrian Collision Warning (PCW)	6
4.1.2. Green Light Optimized Speed Advise (GLOSA)	8
4.1.3. Red Light Violation (RLV)	10
4.1.4. Time-To-Green (TTG)	11
4.1.5. Wrong Way Road Warning (WWR)	12
4.2. V2M Applications	14
4.2.1. Backward Collision Warning (BCW)	14
4.2.2. Turning Vehicle Ahead (TVA)	15
4.2.3. Intersection Movement Assist (IMA)	18
4.2.4. Vulnerable Road User Lane Change Assist (VRU-LCA)	20
4.2.5. Parked Vehicle Leaving (PVL)	22

List of Figures

1. V2V communication between Commsignia devices	1
2. I2V communication between Commsignia devices	1
3. M2V communication between Commsignia devices	1
4. I2M communication between Commsignia devices	2
5. V2M communication between Commsignia devices	2
6. Logic of speed-dependent behavior	5
7. PCW safety application	6
8. GLOSA safety application	8
9. RLV safety application	10
10. TTG safety application	11
11. WWR safety application	13
12. BCW safety application	14
13. TVA safety application in the same lane	16
14. TVA safety application in opposite lanes	16
15. IMA safety application	18
16. VRU-LCA safety application	20
17. PVL safety application	22

1. Introduction

The Safety Applications described here, are solutions provided by Commsignia for the most common real-life V2X use cases that enhance the traveling experience of all road users by making private and public transportation safer, more efficient, and economically sustainable.

Safety applications can be categorized by the connected traffic entities and the direction of the connection. These categories can be as follows:

- **Vehicle-to-vehicle (V2V) applications:** communication between vehicles (such as cars, emergency vehicles, public transportation, and even trains). Figure 1 shows V2V communication between the respective Commsignia devices.

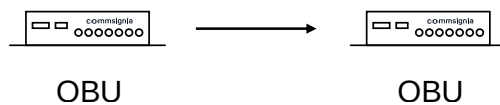


Figure 1. V2V communication between Commsignia devices

- **Infrastructure-to-vehicle (I2V) applications:** vehicular communication between V2X equipped parts of the road infrastructure (such as Traffic Light Controllers in intersections or programmable roadside information signs along a highway) and vehicles. Figure 2 shows I2V communication between the respective Commsignia devices.

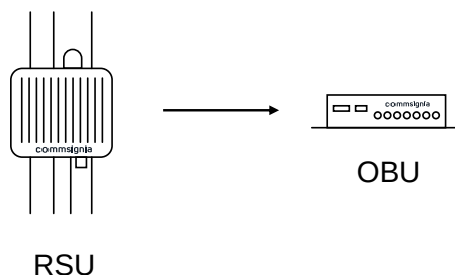


Figure 2. I2V communication between Commsignia devices

- **Micromobility-to-vehicle (M2V) applications:** communication from micromobility devices (such as bicycles, scooters, e-bikes, electronic and mobility scooters, and other light vehicles) to vehicles. Figure 3 shows M2V communication between the respective Commsignia devices.

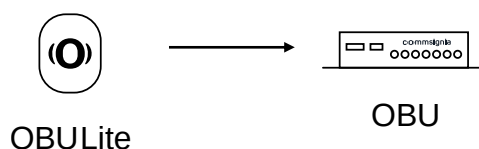


Figure 3. M2V communication between Commsignia devices

- **Infrastructure-to-micromobility (I2M) applications:** vehicular communication between V2X equipped parts of the road infrastructure. Figure 4 shows I2M communication between the respective Commsignia devices.

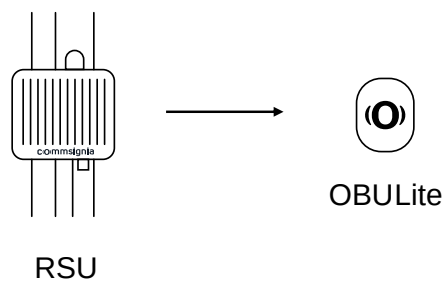


Figure 4. I2M communication between Commsignia devices

- **Vehicle-to-micromobility (V2M) applications:** communication from vehicles to micromobility devices. Figure 5 shows V2M communication between the respective Commsignia devices.

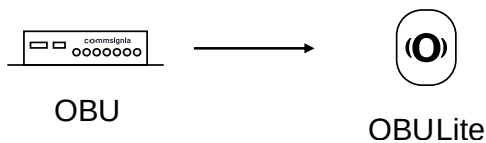


Figure 5. V2M communication between Commsignia devices

This document contains information on micromobility safety applications, I2M and V2M. Each use case is described in detail with their filtering and triggering conditions, and configurable parameters.

This document contains information on car-related safety applications, V2V, V2I, and M2V. Each use case is described in detail with their filtering and triggering conditions, and configurable parameters.

2. Common notification contents

The data fields detailed in this chapter are commonly used by application notifications.

2.1. Object type

The object type is an enumerated type with the following elements:

- Unknown
- Vehicle
- Pedestrian
- Cyclist
- Animal
- Other

2.2. Vehicle type

Object types of Vehicle enumerators are further divided into the following elements:

- Unknown
- PassengerCar
- Special
- LightTruck
- Truck
- Motorcycle
- Transit
- FixedGuideway
- Emergency
- Infrastructure
- Other

2.3. Object data

Object Data represents an object in the global coordinate space. The data consists of the following fields:

- The position of the object, in latitude/longitude format, with a confidence ellipse.
- The speed of the object, with a confidence interval.
- The heading of the object, with a confidence interval.
- The type of the object (see Object Type).
- The vehicle type of the object, if it is a vehicle (see Vehicle Type).

2.4. Relative data

The Relative Data container represents the relative position and motion of an object, compared to another object (usually the host vehicle). The data consists of the following fields:

- The relative position of the object, in X/Y format, with a confidence interval. X is the left–right and Y is the forward–backward axis.
- The distance of the object.
- The relative speed of the object, with a confidence interval.
- The relative heading of the object, with a confidence interval.

3. Speed-dependent behavior

Specific applications can be configured to be automatically enabled or disabled based on the speed of the host vehicle.

3.1. Speed-dependent configuration

These applications have the following configuration values under their own configuration key:

- vMin: The minimum speed where the application is always enabled;
- vMax: The maximum speed where the application is always enabled;
- vMinHyst: The hysteresis of the minimum speed;
- vMaxHyst: The hysteresis of the maximum speed.

3.2. Logic of speed-dependent behavior

Based on the above values, the applications are automatically enabled/disabled, as shown in Figure 6, as follows:

- If the speed of the host vehicle is unknown, the application is disabled.
- If the application is currently enabled and the speed of the host vehicle becomes less than vMin, it is disabled.
- If the application is currently enabled and the speed of the host vehicle becomes greater than vMax + vMaxHyst, it is disabled.
- If the application is currently disabled and the speed of the host vehicle becomes greater than vMin + vMinHyst, it is enabled.
- If the application is currently disabled and the speed of the host vehicle becomes less than vMax, it is enabled.

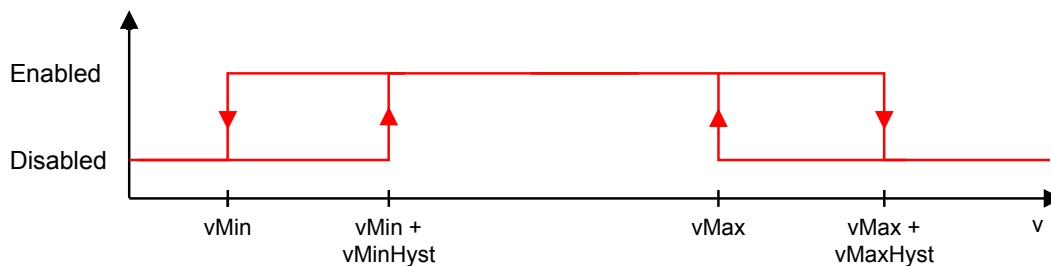


Figure 6. Logic of speed-dependent behavior

4. Safety applications

4.1. I2M Applications

4.1.1. Pedestrian Collision Warning (PCW)

4.1.1.1. Application overview

The PCW safety application is a I2M safety feature that issues a warning to the micromobility user if there is an object ahead in the same lane that is classified as pedestrian, as shown in Figure 7. This application is triggered based on the lane information of the surrounding area, the path information of the micromobility device, and the location of the pedestrian. Pedestrian information (for example their location, their classification as pedestrian, or their projected path of travel) can be received from nearby sensors or mobile devices carried by the pedestrians.

The PCW helps micromobility users to prevent or mitigate the risk of accidents involving micromobility-to-pedestrian collisions, increasing safety, and reducing traffic congestion. This application is compliant and relies on the European (ETSI G5) and the US (WAVE) standards.

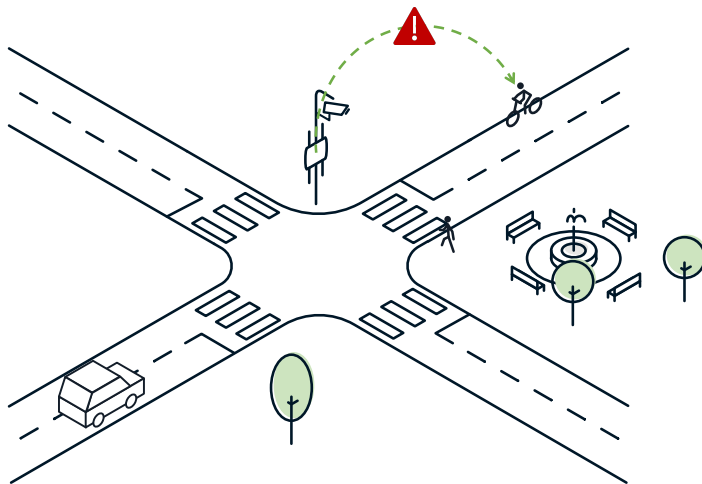


Figure 7. PCW safety application

4.1.1.2. PCW mode of operation

Model

- The PCW is an I2M safety application that is enabled or disabled based on the speed of the host micromobility device (for more information, see "[Speed-dependent behavior](#)" [5]).
- Based on its configuration, the `targetLength` value can be changed based on the speed of the micromobility device. In this case the effective `targetLength` will be equal to `targetLength × (host speed [km/h]/50 [km/h])`. This value needs to be in the range of 10–1000 m.
- If enabled, the maximum time-to-collision (TTC) value can be changed based on the speed of the host micromobility device. The effective maximum TTC is equal to `maximum TTC × (host speed [km/h]/50 [km/h])`. If this computed effective value is less than 3 s, then it is set to be equal to 3 s.

	The application uses the <i>Object</i> entity type.
Filter setup	The application subscribes to remote <i>Objects</i> which match the following criteria: - The remote <i>Object</i> is <i>Pedestrian</i> type. - The pedestrian is at most <code>targetLength</code> distance ahead.
Triggering conditions (normal mode)	If simple mode is not selected, on every processing cycle, for every filtered object: - The application computes the closest it will get to the pedestrian, assuming both are moving on a straight line, at either constant speed or constant acceleration depending on the configuration. - If this distance is less than the configured minimum clearance distance and the time it will take to reach this distance is below the configured maximum TTC, a notification is sent.
Triggering conditions (simplified mode)	If simple mode is selected, on every processing cycle, for every filtered object: - The application computes the time it would take for the host vehicle to collide with the pedestrian, assuming the following: - The pedestrian stays stationary (even if the received data indicates nonzero speed or acceleration). - The host micromobility device continues to move in a straight line with constant velocity. - If the computed TTC is less than the configured threshold, a notification is sent.

4.1.1.3. PCW configuration parameters

The following parameters must be set in the configuration space of `saf.json`:

- The application is enabled by `enable.pcw: true`
- The following parameters can be configured in the `pcw` key:
 - `targetLength`: Length of the target area in [m].
 - `useAdaptiveTarget`: Use speed-dependent target length scaling.
 - `dangerousDistance`: The minimum clearance distance that is considered safe.
 - `ttc`: The maximum time it can take to reach the dangerous distance.
 - `useAdaptiveTtc`: Use speed-dependent TTC scaling. It is recommended to set either this or `useAccelerationForTtc` to `true`.
 - `useAccelerationForTtc`: Assume constant acceleration when calculating the minimum distance point. It is recommended to set either this or `useAdaptiveTtc` to `true`.
 - `useSimpleAlgorithm`: Use a simplified algorithm that assumes all pedestrians are stationary.

4.1.1.4. PCW notifications

The application sends ALERT level V3 notifications which contain the *Object Data* of the host micromobility device and the *Object Data* and *Relative Data* of the remote entity (for more information, see “Common notification contents” [3]).

The application also sends TTC information in the notification as well. This is the predicted time (in ms) until the two entities are closest to each other.

4.1.2. Green Light Optimized Speed Advise (GLOSA)

4.1.2.1. Application overview

The GLOSA is a use case of the Traffic Signal Information (TSI) safety application, which is an I2M safety feature, that suggests a calculated speed to the micromobility user, allowing them to pass through an intersection during the green signal interval, as shown in Figure 8. This information is calculated based on the received TSI and the current speed and location of the micromobility device.

The GLOSA helps micromobility users to prevent or mitigate delays as a result of stop-and-go movements through intersections and optimizing efficiency. This application is compliant and relies on to the European (ETSI G5) and the US (WAVE) standards.

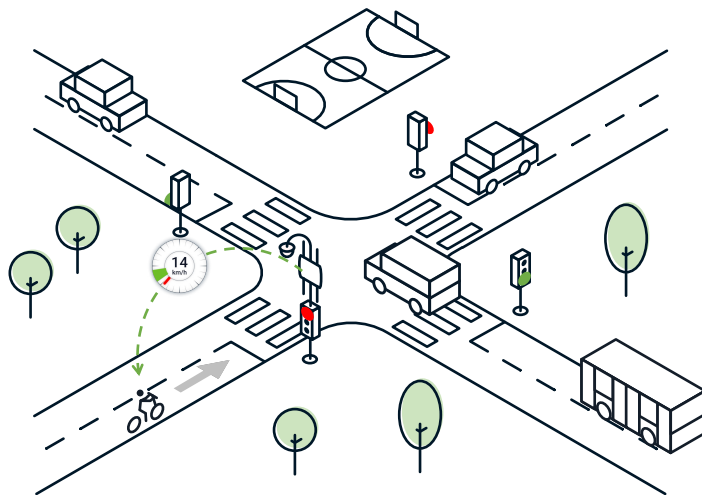


Figure 8. GLOSA safety application

4.1.2.2. GLOSA mode of operation

Model	• TSI is an I2M application that uses traffic signal information received from intersections. All TSI use cases work optimally when future traffic signal phase information is also known. • TSI uses <i>lane abstraction</i> for the target area.
Filter setup	The application subscribes to Intersection Information, where the host micromobility device is in an ingress lane and it is not traveling in the wrong direction.
Triggering conditions	The TSI application handles the following calculations on every processing cycle: • The following data is calculated: ◦ The distance to be traveled to the stop line of the ingress lane.

- The timestamp when the stop line will be reached, assuming constant velocity.
- If the `useTurnSignal` option is on, connections that are not predicted to be taken based on the turn signal status of the host micromobility device are skipped.
- If the micromobility device is traveling too slow or too fast compared to the advised speeds (see below), the connection is added to the GLOSA list.
- If the GLOSA use case is enabled, and the GLOSA list is not empty, a GLOSA notification is sent.
- If both `likelyTime` and `minEndTime` fields are available for a phase, the `likelyTime` field is used if the `preferLikelyTime` option is enabled, otherwise the `minEndTime` is used. If only one of these is available, that field is used, regardless of the configuration.
- The actual GLOSA value for each connection is calculated as follows:
 - Based on the start and stop times of the current or next green light and the distance to the stop line, a min and max advised speed is calculated.
 - If the vehicle is traveling too fast, the advice is set to the calculated minimum speed. If this speed is less than 20 km/h, it is set to zero.
 - If the vehicle is traveling too slow, the advice is set to the calculated maximum speed. If this speed is more than any of the following, the advice is set to zero.
 - Speed limit of the lane
 - 1.5 times the current speed
 - 70 km/h

4.1.2.3. GLOSA configuration parameters

The following parameters must be set in the configuration space of `saf.json`:

- The TSI application is enabled if any of its use cases are enabled, such as `enable.glo: true`
- The following parameter can be configured in the `tsi` key:
 - `preferLikelyTime`: Use the `likelyTime` field of the traffic signal information instead of the `minEndTime` for phase end time calculation, if both are present.

4.1.2.4. GLOSA notifications

GLOSA is always an INFO level notification.

The notification contains the *Object Data* of the host micromobility device (for more information, see ["Common notification contents" \[3\]](#)).

The notification also contains the lane ID of the ingress lane as well as the distance to the stop line of the ingress lane.

The GLOSA notification contains the speed advises with the signal group IDs and the speed advice itself.

4.1.3. Red Light Violation (RLV)

4.1.3.1. Application overview

The RLV is a use case of the Traffic Signal Information (TSI) safety application, which is an I2Msafety feature that provides a warning when the micromobility vehicle is about to run a red light at an intersection. This information is based on the received Traffic Signal Information.

The RLV helps drivers to prevent hazards at intersections resulting from running a red light. This application is compliant and relies on to the European (ETSI G5) and the US (WAVE) standards.

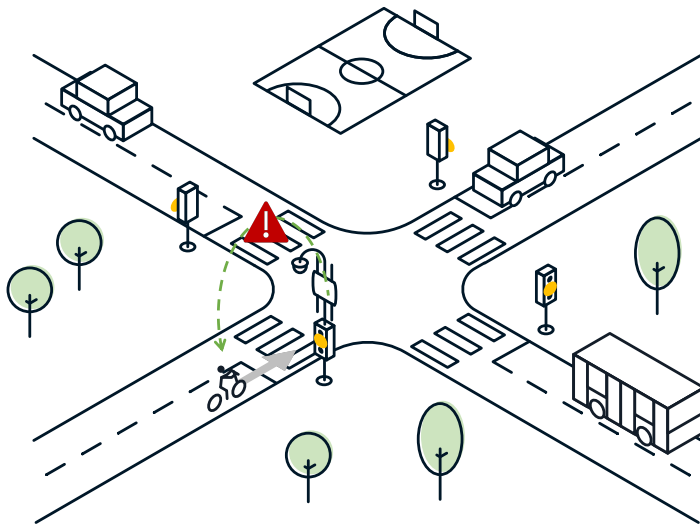


Figure 9. RLV safety application

4.1.3.2. RLV mode of operation

Model	• TSI is an I2M application that uses traffic signal information received at intersections. All TSI use cases work optimally when future traffic signal phase information is also known. • TSI uses <i>lane abstraction</i> for the target area.
Filter setup	The application subscribes to the <i>Intersection</i> Information, where the micromobility vehicle is in an ingress lane and traveling in the appropriate direction.
Triggering conditions	The TSI application handles the following calculations on every processing cycle: • The following data is calculated: ◦ The distance to be traveled to the stop line of the ingress lane. ◦ The timestamp when the stop line will be reached, assuming constant velocity. • If the current phase is red, and the end time is after the stop line timestamp, the connection is added to the RLV list. • If the current phase is not red, the start and end times of the next red phase are available, and the stop line timestamp is within this interval, the connection is added to the RLV list.

- If the RLV use case is enabled, and the RLV list is not empty, a RLV notification is sent.
- If both the `likelyTime` and the `minEndTime` fields are available for a phase, the `likelyTime` field is used if the `preferLikelyTime` option is enabled, otherwise `minEndTime` is used. If only one of these is available, that field is used, regardless of the configuration.

4.1.3.3. RLV configuration parameters

The following parameters must be set in the configuration space of `saf.json`:

- The TSI application is enabled if any of its use cases are enabled such as `enable.rlv: true`.
- The following parameter can be configured in the `tsi` key:
 - `preferLikelyTime`: Use the `likelyTime` field of the traffic signal information instead of `minEndTime` for phase end time calculation, if both are present.

4.1.3.4. RLV notifications

RLV is always an ALERT level notification.

The notification contains the *Object Data* of the host vehicle (for more information, see “[Common notification contents](#)” [3]) and the *Object Data* and *Relative Data* of the remote vehicle.

The notification also contains the lane ID of the ingress lane as well as the distance to the stop line of the ingress lane.

The RLV notification contains the red light information with the signal group IDs and the start and end timestamps of the red phase.

4.1.4. Time-To-Green (TTG)

4.1.4.1. Application overview

The TTG is a use case of the Traffic Signal Information (TSI) safety application, which is a I2M safety feature that provides information about the time remaining until the next green signal interval of the traffic light ahead within the same lane. This information is based on the received Traffic Signal Information.

TTG helps micromobility users to prevent or mitigating delays as a result of stop-and-go motions through intersections and optimizing their efficiency. This application is compliant and relies on to the European (ETSI G5) and the US (WAVE) standards.

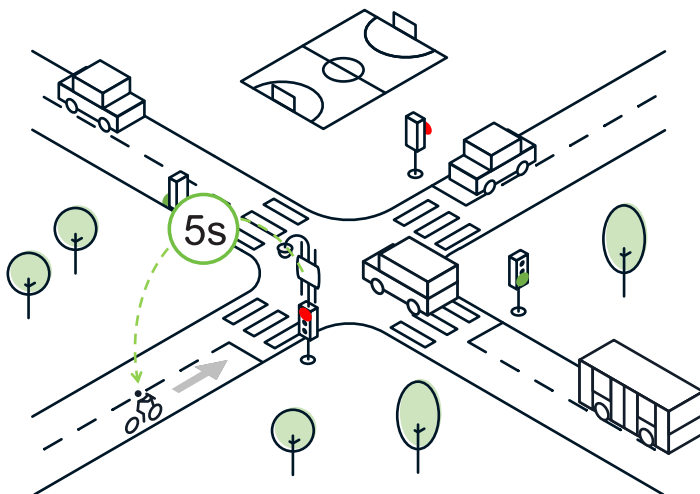


Figure 10. TTG safety application

4.1.4.2. TTG mode of operation

Model	• TSI is an I2M application that uses traffic signal information received at intersections. All TSI use cases work best when future traffic signal phase information is also known. • TSI uses [lane abstraction] for the target area.
Filter setup	The application subscribes to the <i>Intersection</i> Information, where the micromobility vehicle is in an ingress lane and it is traveling in the appropriate direction.
Triggering conditions	The TSI application handles the following calculations on every processing cycle: • The following data is calculated: ◦ The distance to be traveled to the stop line of the ingress lane. ◦ The timestamp when the stop line will be reached, assuming constant velocity. • For each remaining connection, if the current phase is not green, and the start time of the next green phase is available, the connection is added to the TTG list. • If the TTG use case is enabled, and the TTG list is not empty, a TTG notification is sent. • If both the <code>likelyTime</code> and the <code>minEndTime</code> fields are available for a phase, the <code>likelyTime</code> field is used if the <code>preferLikelyTime</code> option is enabled, otherwise <code>minEndTime</code> is used. If only one of these is available, that field is used, regardless of the configuration.

4.1.4.3. TSI - TTG configuration parameters

The following parameters must be set in the configuration space of `saf.json`:

- The TSI application is enabled if any of its use cases are enabled such as `enable.ttg: true`
- The following parameter can be configured in the `tsi` key:
 - `preferLikelyTime`: Use the `likelyTime` field of the traffic signal information instead of `minEndTime` for phase end time calculation, if both are present.

4.1.4.4. TTG notifications

TTG is always an INFO level notification.

The notification contains the Object Data of the host vehicle (for more information, see ["Common notification contents" \[3\]](#)) and the Object Data and Relative Data of the remote vehicle.

The notification also contains the lane ID of the ingress lane as well as the distance to the stop line of the ingress lane.

The TTG notification contains the speed advises with the signal group IDs and the time to green in ms.

4.1.5. Wrong Way Road Warning (WWR)

4.1.5.1. Application overview

The WWR is a use case of the Traffic Information Warning (TIW) I2M safety application that provides a notification message to the micromobility user, received from the road infrastructure about entering

the given road segment the wrong way (normally conveyed through static or variable message signs along the road).

The WWR helps micromobility users to receive information and reminders about wrong way entries, thus reducing the risk of traffic hazards. This application is compliant and relies on the US (WAVE) standard.

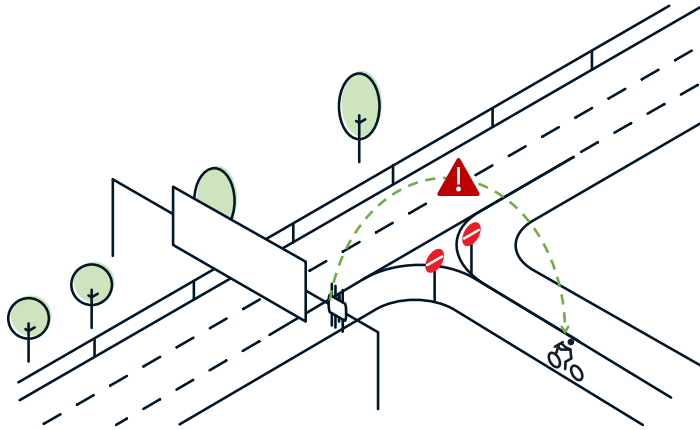


Figure 11. WWR safety application

4.1.5.2. WWR mode of operation

Model	- The WWR is a I2M application use case that works for the <code>TrafficInfo</code> remote entity type. - The application uses object-in-area algorithms for filtering. - The triggering conditions of the application are based on [ITIS codes] and the configured speeding threshold.
Filter setup	The TIW application subscribes to <i>Road Information</i>, where: - The micromobility vehicle must be in the [relevance zone] of the <code>TrafficInfo</code> entity type. The relevance zone can be either a designated track or a circular area. - The host vehicle must also be at the [same altitude] as the [relevance zone] of the <code>TrafficInfo</code> entity type. - The heading of the host vehicle must be [aligned with the zone's direction]. In the WWR use case the heading is the "wrong way" relative to the zone's direction.
Triggering conditions	- The application is triggered based on ITIS codes - If the "Vehicle Travelling Wrong Way" ITIS code is found anywhere in the Road Information, then a WWR use case is triggered in TIW. - If TIW is triggered by a filtered object with Road Information, but none of the use cases match any triggering criteria, then a general TIW notification is triggered.

4.1.5.3. TIW and WWR configuration parameters

The following parameters must be set in the configuration space of `saf.json`:

- The application is enabled by `enable.wwr: true`
- The generic TIW application and the WWR use case have no additional configuration.

4.1.5.4. WWR and general TIW notifications

The TIW application displays different notification levels based on the [TrafficInfo] type and the speeding threshold. In the WWR use case, the notification level is always ALERT. The generic TIW different notification levels are the following:

- [Alert] - this notification is displayed when the host vehicle is speeding within a [Regulation] or [Danger] type.
- [Warning] - this notification is displayed when the host vehicle is speeding within any other type, for example [Advice].
- [Info] - this is an informative notification, displayed when the host vehicle is not speeding.

The notification contains the Object Data of the host vehicle (for more information, see “[Common notification contents](#)” [3]) and the Object Data and Relative Data of the remote vehicle.

General TIW notifications also contain the use case, if multiple cases are triggered then the latest one.

TIW notifications also contain the ITIS code list of the Road Information object.

4.2. V2M Applications

4.2.1. Backward Collision Warning (BCW)

4.2.1.1. BCW safety application overview

The BCW is a safety application that issues a warning to the micromobility user of the host vehicle in case of an impending rear-end collision with a remote vehicle behind in traffic and in the direction of travel.

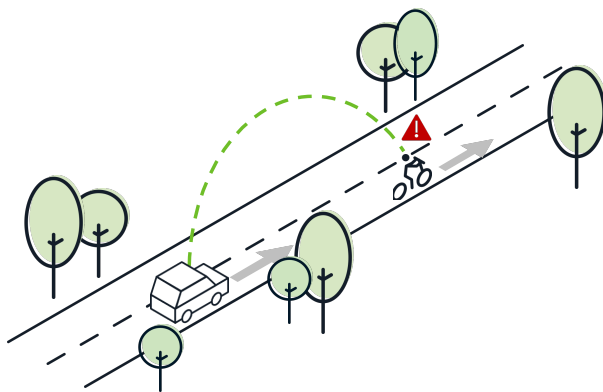


Figure 12. BCW safety application

4.2.1.2. Mode of operation

- | | |
|-------|--|
| Model | <ul style="list-style-type: none"> • The BCW application is a V2M application, that works for the <i>Object</i> remote entity type • The application is enabled or disabled based on the speed of the host vehicle. • The application uses [polygon] classification for target area behind the vehicle. |
|-------|--|

	The application is triggered based on the speed difference of the entities.
Filter setup	- The vehicles are <u>behind the VRU, at the same altitude, and traveling in the same direction.</u> - The vehicles are inside the target area defined by the following criteria: The <u>backward</u> dimension is set by <code>lengthBehind</code>, while the <u>to the left</u> and the <u>to the right</u> dimensions are set by <code>widthLeft</code> and <code>widthRight</code>, respectively.
Triggering conditions	- The actual speed of the host vehicle is a runtime value and is not set by the configuration. <u>Area</u> is defined as a rectangle behind the host VRU with a side length of <code>widthLeft + widthRight</code>. - The application is triggered if the approaching vehicle is in the area and it is much faster than the host VRU, the distance is under <code>dangerousDistance</code>, and the time-to-collision is under is less than <code>ttcThr</code>.

4.2.1.3. Configuration parameters

The following parameters must be set in the configuration space of `saf.json`:

- The application is enabled by `enable.bcw: true`
- The following parameters can be configured in the **bcw** member
 - Speed-dependent configuration [m/s] as described in section [“Speed-dependent behavior” \[5\]](#).
 - Polygon filter configuration [m]
 - `lengthBehind`: The base length of the target area backwards.
 - `widthRight`: The base width of the target area on the right.
 - `widthLeft`: The base width of the target area on the left.
 - `dangerousDistance`: The distance between the host and the remote considered dangerous [m].
 - `ttcThr`: The time between the host and the remote that is considered dangerous [s].

4.2.1.4. Notification parameters and descriptions

The application sends ALERT level V3 notifications which contain the Object Data of the host vehicle and the Object Data and Relative Data of the remote vehicle (for more information, see [“Common notification contents” \[3\]](#)). The name of the use-case is BCW for backward compatibility reasons; it is converted to V2 GCW notification in the case of previous API versions.

The application also sends Time-to-Collision information in the notification as well. This is the predicted time (in ms) until the two vehicles are closest to each other.

4.2.2. Turning Vehicle Ahead (TVA)

4.2.2.1. Application overview

The TVA warns the micromobility user about the possible dangers of surrounding vehicles intending to cross the road in the path of the user. The remote vehicle approaches from the same or from the opposite direction using its turn signal corresponding to the direction of the vulnerable road user.

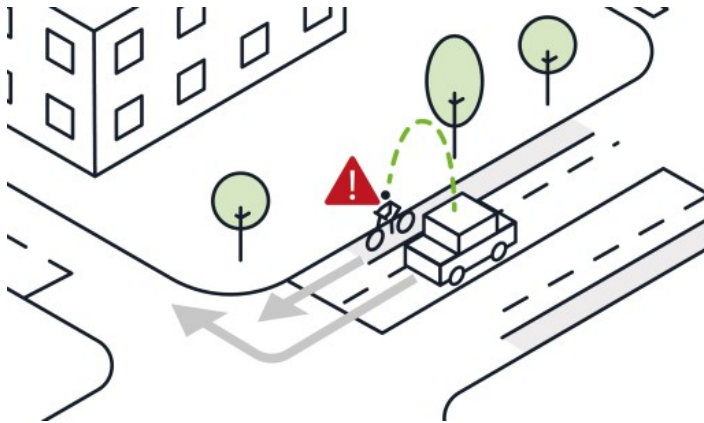


Figure 13. TVA safety application in the same lane

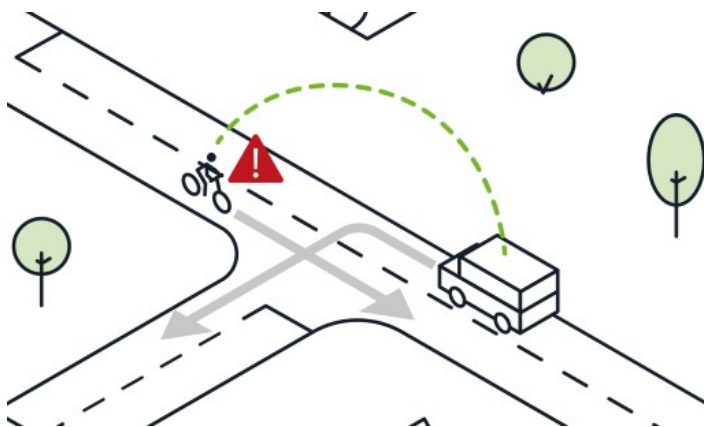


Figure 14. TVA safety application in opposite lanes

4.2.2.2. Mode of operation

Model	- The TVA application is a V2M application, that works for the <i>Object</i> remote entity type. - The application is enabled or disabled based on the speed of the host vehicle. - The application uses <code>polygon</code> classification for target area on the right. - The application is triggered based on the existence of the remote vehicle. - Precondition: the turn signal of the remote vehicle — corresponding to the relative direction of the road user — is turned on.
Filter setup	- The direction of the remote vehicle is parallel with the movement of the micromobility vehicle. - The remote vehicle is <u>in the left</u> area or the <u>right area</u> - The <u>area of turn</u> is defined as a rectangles in the turn direction ahead and behind of the host vehicle with a side length of <code>widthLeft</code> or <code>widthRight</code>.

- The remote vehicle is approaching:
 - from ahead, and the remote vehicle is inside the target area defined by the following criteria: The ahead dimension is set by `lengthAhead` and the backward dimension is set to `zero`.
 - from behind, and the remote vehicle is inside the target area defined by the following criteria: The ahead dimension is set by `lengthAhead` and the backward dimension is set by `lengthBehind`.
- The remote vehicle is at the same altitude.

Triggering conditions

- The application is triggered based on the existence of a remote vehicle in the area of interest while using its turn signal corresponding to the relative direction of the vulnerable road user.
- The turn is considered dangerous if the estimated time-to-collision is below the `ttcThr`.

4.2.2.3. Configuration parameters

The following parameters must be set in the configuration space of `saf.json`:

- The application is enabled by `enable.tva: true`
- The following parameters can be configured in the `tva` member
 - Speed-dependent configuration [m/s] as described in section “[Speed-dependent behavior](#)” [5].
 - Polygon filter configuration [m]:
 - `lengthAhead`: The base length of the target area ahead in [m].
 - `lengthBehind`: The base length of the target area backwards in [m].
 - `widthRight`: The base width of the target area on the right in [m].
 - `widthLeft`: The base width of the target area on the left in [m].
 - `ttcThr`: Time margin for a safe turn in [s].

Common configuration parameters affecting the application in `cff.json`:

- Same direction classifier configuration in `core.classification.headingDiffThr` member.
- Same altitude classifier configuration in `core.classification.altitudeDiffThr` member.

4.2.2.4. Notification parameters and descriptions

The application sends ALERT level V3 notifications which contain the Object Data of the host vehicle and the Object Data and Relative Data of the remote vehicle (for more information, see “[Common notification contents](#)” [3]). The name of the use case is TVA for backward compatibility reasons; it is converted to V2 LTA/RTA notification in the case of previous API versions, based on the direction of the turn signal.

The application also sends Time-to-Collision information in the notification as well. This is the predicted time (in ms) until the two vehicles are closest to each other.

4.2.3. Intersection Movement Assist (IMA)

4.2.3.1. Application overview

The IMA application warns the micromobility user in case of an impending side-to-front/front-to-side collision with a remote vehicle traveling in a direction perpendicular to the micromobility vehicle, typically in an intersection.

The IMA helps drivers to prevent or mitigate side-to-front or front-to-side vehicle collision in the forward path of travel.

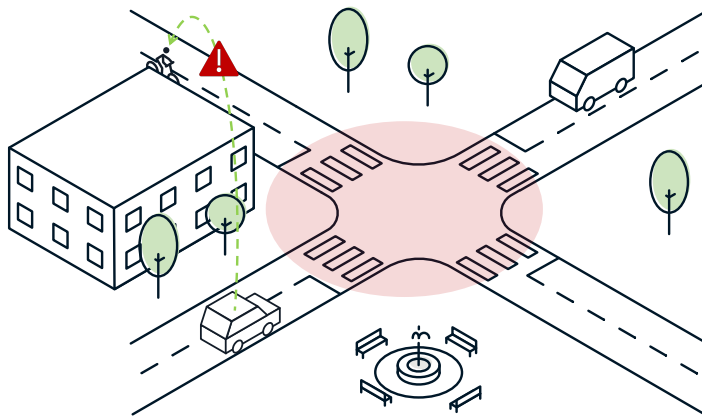


Figure 15. IMA safety application

4.2.3.2. Mode of operation

Model

- The IMA application is a V2M application, that works for the *Object* remote entity type
- IMA is enabled or disabled based on the speed of the host vehicle (for more information, see ["Speed-dependent behavior" \[5\]](#)).
- If enabled, the `targetLength` value can be changed based on the speed of the ego vehicle. The effective `targetLength` is equal to $\text{targetLength} \times (\text{host speed [km/h]} / 50 \text{ [km/h]})$. This value must be in the range of 10–1000 m.
- If enabled, the maximum TTC value changes based on the speed of the ego vehicle. The effective maximum TTC is equal to $\text{maximum TTC} \times (\text{host speed [km/h]} / 50 \text{ [km/h]})$. If this computed effective value is less than 3 s, then it is set to be equal to 3 s.
- The application uses distance based filtering for target area.
- The application is triggered based on the minimum predicted distance of objects.
- Precondition for the application: the remote object is not stationary. The application uses a predefined speed limit to decide whether or not the object is stationary.

Filter setup

- The remote vehicle is in the range of `targetRadius`. This can optionally be set to adapt target radius for host speed, using `useAdaptiveTarget`.

- The remote vehicle is approaching from side direction.
- The remote vehicle is located ahead.
- The remote vehicle is in the same altitude.
- The remote vehicle is moving with at least 5 km/h.

Triggering conditions

On every processing cycle, for every filtered object:

- The application computes the closest it will get to the object, assuming both are moving on a straight line, at either constant speed or constant acceleration depending on the configuration.
- The calculated closest distance is corrected using the length of both objects.
- If this distance is less than the configured minimum clearance distance and the time it will take to reach this distance is below the configured maximum TTC, a notification is sent

4.2.3.3. Configuration parameters

The following parameters must be set in the configuration space of `saf.json`:

- The application is enabled by `enable.ima: true`
- The following parameters can be configured in the `ima` member
 - Speed-dependent configuration [m/s]
 - `vMin`: The minimum speed where the application is always enabled.
 - `vMax`: The maximum speed where the application is always enabled.
 - `vMinHyst`: The hysteresis of the minimum speed.
 - `vMaxHyst`: The hysteresis of the maximum speed.
 - The description of the logic of speed-dependent behavior can be found in section ["Speed-dependent behavior" \[5\]](#).
 - `targetRadius`: The base radius of the target area in [m].
 - `useAdaptiveTarget`: Adapt effective `targetRadius` in proportion to the speed of the host vehicle.
 - `dangerousDistance`: The distance of the remote vehicle to be considered [dangerous], in [m].
 - `ttc`: Time-to-collision (TTC) threshold, in [s].
 - `useAdaptiveTtc`: Adapt effective TTC proportionally to the speed of the host vehicle.
 - `accelerationForTtc`: Use host and remote vehicle acceleration data when calculating TTC.

Common configuration parameters affecting the application in `cff.json`:

- [From side] classifier configuration in `core.classification.headingDiffThr`
- [Same altitude] classifier configuration in `core.classification.altitudeDiffThr`

4.2.3.4. IMA Notifications

The application sends ALERT level notifications which contain the Object Data of the host vehicle and the Object Data and Relative Data of the remote vehicle (for more information, see "[Common notification contents](#)" [3]).

The application also sends Time to Collision information in the notification as well. This is the predicted time (in ms) until the two vehicles are closest to each other.

4.2.4. Vulnerable Road User Lane Change Assist (VRU-LCA)

4.2.4.1. Application overview

The VRU-LCA warns the micromobility user about the possible dangers of changing lanes when the road ahead is blocked by another vehicle. The warning is provided to prevent a collision with remote vehicles in the intended destination lane of the host micromobility vehicle. The application operates using a target area and various filtering and triggering criteria as well. The application is active if the host is within the specified speed range.

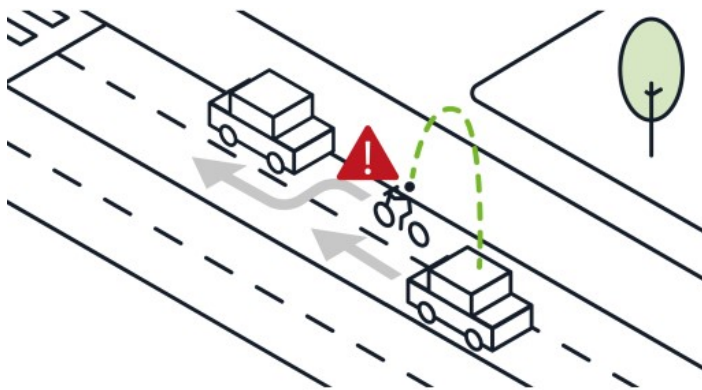


Figure 16. VRU-LCA safety application

4.2.4.2. Mode of operation

Model	• The VRU-LCA application is a V2M application, that works for the <i>Object</i> remote entity type. • The application is enabled or disabled based on the speed of the host vehicle. • The application uses <code>polygon</code> classification for target area on the right. • The application is triggered based on the time required to complete a lane change and the time for an approaching vehicle to reach the VRU. • Precondition: a stationary vehicle is present ahead of the VRU in the target area.
Filter setup	• The vehicles are approaching <u>on the left</u>. • The stationary vehicle is <u>in front</u> of the host. • The approaching vehicles are inside the target area defined by the following criteria: The <u>backward</u> dimension is set by <code>lengthBehind</code> and the width of the target area to the left is set by <code>widthLeft</code>.

	• The stationary vehicle is inside the target area defined by the following criteria: The <u>ahead</u> dimension is set by <code>lengthAhead</code> and the width of the target area to the right is set by <code>widthRight</code>. • The remote vehicle is at the <u>same altitude</u>.
Triggering conditions	• The estimated collision time is proportional to the distance between the micromobility and the approaching vehicles divided by the speed of the approaching vehicle. • The actual speed of the host vehicle is a runtime value and it is not set by the configuration. • The application is triggered if there is at least one vehicle ahead of the micromobility vehicle, the speed of the first vehicle ahead is below the <code>stationarySpeedThr</code>. • The lane change is considered dangerous if the estimated time to enter the range <code>dangerousGapLength</code> from the micromobility vehicle is below the time until the micromobility vehicle is within <code>laneChangeStartGapLength</code> from the stationary vehicle.

4.2.4.3. Configuration parameters

The following parameters must be set in the configuration space of `saf.json`:

- The application is enabled by `enable.lca: true`
- The following parameters can be configured in the `lca` member
 - Speed-dependent configuration [m/s], as described in section “Speed-dependent behavior” [5].
 - Polygon filter configuration [m].
 - `lengthAhead`: The base length of the target area ahead in [m].
 - `lengthBehind`: The base length of the target area backwards in [m].
 - `widthRight`: The base width of the target area on the right in [m].
 - `widthLeft`: The base width of the target area on the left in [m].
 - `laneChangeStartGapLength`: The distance in [m] between the remote micromobility vehicle and the stationary vehicle at the beginning of the lane change.
 - `dangerousGapLength`: The distance in [m] between the host vehicle and the micromobility vehicle from which the situation is considered as dangerous.
 - `ttcThr`: The time threshold for the `time-to-collision` below which the situation is considered as dangerous in [s].

Common configuration parameters affecting the application in `cff.json`:

- Same direction classifier configuration in `core.classification.headingDiffThr` member.
- Same altitude classifier configuration in `core.classification.altitudeDiffThr` member.
- Same speed classifier configuration in `core.classification.sameSpeedEps` member.

4.2.4.4. Notification parameters and descriptions

The application sends ALERT level V3 notifications which contain the Object Data of the host vehicle and the Object Data and Relative Data of the remote vehicle (for more information, see “Common

notification contents" [3]). The name of the use case is VRU-LCA for backward compatibility reasons; it is converted to V2 LCA notification in the case of previous API versions.

The application also sends Time-to-Collision information in the notification as well. This is the predicted time (in ms) until the two vehicles are closest to each other.

4.2.5. Parked Vehicle Leaving (PVL)

4.2.5.1. Application overview

The PVL is a safety application that issues a warning to the micromobility user in case a vehicle is turning out of its parking space in front of it.

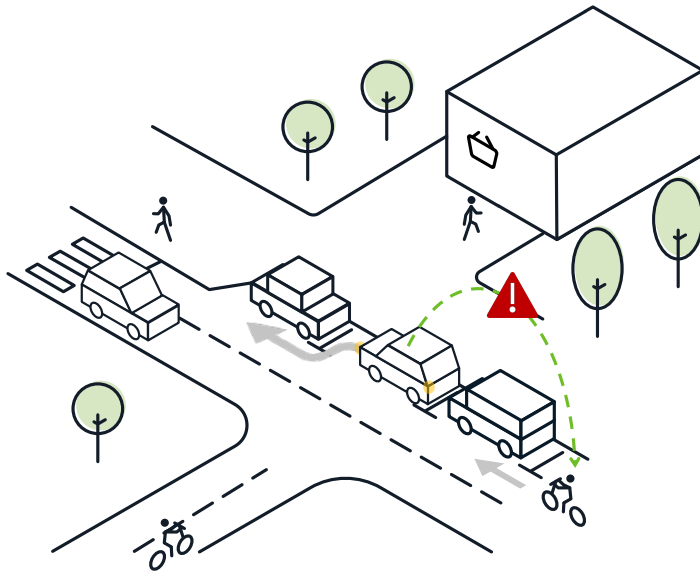


Figure 17. PVL safety application

4.2.5.2. Mode of operation

Model	- The PVL application is a V2M application that works for the <i>Vehicle</i> remote entity type. - The application is enabled or disabled based on speed of the the micromobility user. - The application uses [polygon] classification for the target area. - The application is triggered based on the time required for the micromobility user to reach the position where the remote vehicle turns out of the parking space. - Precondition: The left turn signal of the remote vehicle is turned on.
Filter setup	- The remote vehicles are inside the configured polygon, defined by the following criteria: The <u>ahead</u> dimension is set by <code>lengthAhead</code> and the right width is set by <code>widthRight</code>. - The remote vehicle is at the <u>same altitude</u>.
Triggering conditions	The speed of the remote vehicle is lower than the <code>stationary-SpeedThr</code>.

- The estimated collision time is the difference between the time it takes the host micromobility user to reach the remote vehicle.
- The turn is considered dangerous if the estimated time-to-collision is below `ttcThr`

4.2.5.3. Configuration parameters

The following parameters must be set in the configuration space of `saf.json`:

- The application is enabled by `enable.pvl: true`
- The following parameters can be configured in the `pvi` member
 - Speed-dependent configuration [m/s], as described in section “[Speed-dependent behavior](#)” [5].
 - Polygon filter configuration [m]
 - `lengthAhead`: The base length of the target area ahead in [m].
 - `widthRight`: The base width of the target area on the right in [m].
 - `stationarySpeedThr`: The stationary speed threshold [m/s]. Applications will only be triggered if the remote Vehicle is not stationary.
 - `useTurnSignal`: Considering the state of the turn signal of the stationary in calculations [boolean]
 - `ttcThr`: The time threshold for the time-to-collision below which the situation is considered as dangerous in [s].

Common configuration parameters affecting the application in the `cff.json`:

- Same altitude classifier configuration in the `core.classification.altitudeDiffThr` member.

4.2.5.4. Notification parameters and descriptions

The application sends ALERT level V3 notifications which contain the Object Data of the host vehicle and the Object Data and Relative Data of the remote vehicle (for more information, see “[Common notification contents](#)” [3]). The name of the use-case is PVL for backward compatibility reasons; it is converted to V2 LCA notification in the case of previous API versions.

The application also sends Time-to-Collision information in the notification as well. This is the predicted time (in ms) until the two vehicles are closest to each other.